

南開大學

实验报告

实验 03：基于 UDP 服务的可靠传输协议设计与编程实现

姓名：尚文轩

学号：2412456

专业：计算机科学与技术

任课教师：徐敬东、张建忠

2026 年 10 月 22 日

目录

1 实验目标与验证范围 **2**

2 系统结构与运行环境 **2**

3 协议语法与字段定义 **3**

3.1 基本参数与 48 字节首部 3

3.2 消息类型与正文 4

3.3 差错检测 4

4 连接管理与时序 **5**

4.1 建立连接 5

4.2 关闭与异常 5

5 固定窗口、选择确认与重传 **6**

5.1 序号空间不变式 6

5.2 累计确认与位图映射 6

5.3 逐段超时与接收缓存 6

6 实现结构与核心方法 **7**

6.1 基础 Socket 调用 7

6.2 发送端算法 7

6.3 接收端与提交 8

7 功能与可靠性实测 **8**

7.1 测试方法与覆盖 8

7.2 选择重传的实际日志 9

7.3 控制阶段恢复 10

8 窗口与丢包性能实验 **10**

8.1 配置、计时与单位 10

8.2 实测表与图 11

8.3 结果解释与局限 11

9 问题处理与结论 **12**

参考文献 **12**

1 实验目标与验证范围

本实验以 UDP 数据报套接字为基础，在用户空间实现面向连接的单向可靠文件传输协议 RUDP-SR/1。业务数据仅从发送端流向接收端，握手、确认、关闭与异常通知双向交互。程序使用 C++17 和基础 Winsock API，自行实现连接状态、差错检测、流水线选择确认重传与固定窗口，不使用 CSocket、现成可靠 UDP 或传输框架。

课程《实验说明》第 6 页强调连接管理、校验和、选择确认、同大小固定窗口，以及在规定环境传输给定文件；第 8 页要求运行展示、必要日志和传输协议讲解。该说明及备注中未列出具体文件清单、Router 参数或拓扑，因此本报告**明确采用本机可控模拟链路与自生成文件**。这不能代替尚未提供的课堂指定环境；同一程序支持任意输入文件、显式地址和端口，可在补充要求后重测。

表 1: 实验要求与实现、验证对应

要求	实现	验证
连接管理	建立握手、就绪确认、关闭、TIME_WAIT、RESET	各控制报文定点丢失与超时
差错检测	全首部/正文 Internet 校验和，整文件 CRC32	单比特变异、真实损坏、CRC 失败
流水线与选择确认	ack_base + 64 位 SACK，各段独立计时	丢 DATA 2 时只重传 DATA 2
同大小固定窗口	双方固定 W，握手不一致拒绝	W=1/4/16/64、缓存上界断言
单向文件、双向控制	DATA 向接收端，ACK 等反向	真实 UDP 进程交互
性能与完整交付	时间、有效吞吐率、日志、源码、exe、报告	36 次窗口/丢包实测与 SHA256 复核

2 系统结构与运行环境

实测使用 Windows 11（10.0.26200）、MSVC 19.29 x64、C++17。编译参数包含 `/W4/WX/utf-8/O2/MT`，警告视为错误并采用静态运行库；链接系统 `ws2_32.lib`，运行不依赖 Npcap 或第三方可靠传输库。Python 仅为测试驱动、数据分析及绘图工具，收发与故障转发均由 C++ 实现。

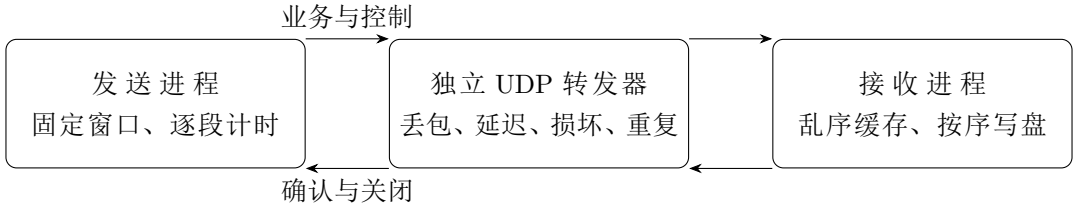


图 1: 性能实验拓扑；也支持不经过转发器的直接传输

转发器使用一个绑定 UDP 端口双向转发，记住发送端地址，并以实际接收端地址识别返回报文。它在应用报文上制造差错，不修复 RUDP 校验和；Windows 随后为新的 UDP 数据报计算传输层封装，因此能验证用户空间校验，而非只依赖内核 UDP 校验。默认收发仅绑定 127.0.0.1；跨主机与外部 Router 可通过 `--bind/--peer/--port/--local-port` 配置。

3 协议语法与字段定义

3.1 基本参数与 48 字节首部

每个会话使用非零 64 位随机 ID 与对端地址共同标识，避免将其他会话或来源的数据混入当前文件。固定窗口 $W \in [1, 64]$；发送端负载大小 L 默认 1200 字节，可设为 1–1400 字节。最大应用数据报为 $48 + 1400 = 1448$ 字节，加 IPv4/UDP 首部后为 1476 字节，通常小于 1500 字节 MTU；特殊网络仍应按现场 MTU 配置。分段号从 0 开始，不回绕；

$$N = \left\lceil \frac{F}{L} \right\rceil \leq 2^{32} - 1.$$

空文件的 $N = 0$，仍正常建立和关闭。所有多字节字段采用网络字节序，显式序列化，不直接发送 C++ 结构体。

表 2: RUDP-SR/1 固定首部

偏移	字节	字段	语义
0	4	magic	`0x52554450`，即 RUDP
4	1	version	固定为 1
5	1	type	类型 1–9
6	2	header_size	固定为 48
8	8	session	非零连接 ID
16	4	seq	DATA 分段号；FIN 为总段数；其他为 0
20	4	ack_base	首个未按序提交的分段号
24	8	sack	bit i 表示 ack_base+i 已缓存
32	2	payload_size	实际数据报长度必须为 48 加此值
34	2	window	固定窗口 W，会话中保持不变
36	2	checksum	首部与正文的 16 位 Internet 校验和
38	2	reserved	必须为 0
40	8	file_size	完整文件字节数，与握手一致

3.2 消息类型与正文

表 3: 消息语义

值	类型	方向	正文与行为
1	SYN	S→R	8 字节：chunk(u16)、0(u16)、CRC32(u32)
2	SYN_ACK	R→S	回显相同参数，拒绝窗口不一致
3	START	S→R	相同 8 字节参数，建立握手末报文
4	DATA	S→R	分段正文，末段长度按文件大小计算
5	ACK	R→S	空，使用首部 ack_base 与 SACK
6	FIN	S→R	CRC32(u32)、总分段数 (u32)，seq=N
7	FIN_ACK	R→S	空，表示文件已校验并提交
8	CLOSE_ACK	S→R	空，确认 FIN_ACK 并进入 TIME_WAIT
9	RESET	双向	可读错误原因，会话失败

除 ACK 外，ack_base 和 sack 必须为 0；除 DATA/FIN 外 seq 必须为 0。`SYN/SYN_ACK/START/FIN` 正文必须为 8 字节，`ACK/FIN_ACK/CLOSE_ACK` 正文必须为空，DATA 不能为空。未知版本、非法长度、保留字段、非法窗口及不符合类型的字段直接丢弃并记录。

3.3 差错检测

校验和使用 RFC 1071 反码求和方法 [1]：将首部及正文按 16 位大端字相加，奇数末字节补 0，折叠高位进位，再按位取反。构造报文时 checksum 字段置 0 后计算；接收时包含 checksum 求和结果应为全 1，即本实现的校验函数返回 0。

```
while (n >= 2) {
    sum += (uint32_t(p[0]) << 8) | p[1];
    p += 2; n -= 2;
}
if (n) sum += uint32_t(p[0]) << 8;
while (sum >> 16) sum = (sum & 0xffff) + (sum >> 16);
return static_cast<uint16_t>(~sum);
```

错误报文不发送肯定 ACK，由发送端独立超时恢复。接收端按顺序写入时另增量计算整文件 CRC32，提交前与握手参数比较。测试驱动再用 SHA256 与逐字节比较独立核对。校验和与 CRC32 均可能碰撞，不能作为身份认证或抗恶意篡改机制。

4 连接管理与时序

4.1 建立连接

建立交互为 SYN、SYN_ACK、START，并增加 ACK(base=0) 作为就绪确认；因此正常发送 DATA 前会看到四条控制报文。接收端只在合法 START 后建立临时文件；重复 SYN 或 START 重发响应，不重置已经收到的数据。发送端对每个控制阶段按 RTO 重试，默认 RTO150ms、最多 40 次重试；接收等待/空闲时限默认 10000ms。



图 2: 建立、单向数据传输与关闭的正常时序

4.2 关闭与异常

发送端只有全部数据确认后才能发送 FIN。接收端检查总段数、累计字节数和 CRC32，刷新并关闭临时文件后将其重命名为最终目标，再发送 FIN_ACK。FIN 或 FIN_ACK 丢失时重复 FIN/FIN_ACK 不再次写入文件。发送端收到 FIN_ACK 后发送 CLOSE_ACK，在 3RTO 的 TIME_WAIT 内对重复 FIN_ACK 再次确认。

每个阶段与数据段都有重试预算。连续丢失超过预算、对端 RESET、写盘失败或中断均返回非零状态；未提交文件由本进程删除，不覆盖已有目标或既有.part。如果接收端已提交文件而 CLOSE_ACK 持续丢失，接收端可能在关闭确认超时后失败，已验证文件仍保留，日志明确提示。这体现了有限重试在永久故障下的状态边界，不能承诺双方对最终状态无限可靠地达成一致。

接收端依次经历`LISTEN`、`SYN_RECEIVED`、`ESTABLISHED`、`FILE_COMMITTED/LAST_ACK`和`CLOSED`。发送端依次经历`SYN_SENT`、`ESTABLISHED`、`FIN_WAIT`、`TIME_WAIT` 和`CLOSED`。Ctrl+C只设置停止标志，通过正常栈展开清理资源；不在控制台回调中写文件或操作复杂共享状态。

5 固定窗口、选择确认与重传

5.1 序号空间不变式

发送端维护最左未确认序号 s 与下一未发送序号 n，必须满足

$$s \le n \le s + W.$$

接收端维护首个未按序提交号 r，只缓存 $[r, r+W)$ 中的段。每个段只能被按序写入一次。发送端不能仅根据“当前未确认数量小于 W”就继续向前发送：如果左侧有缺口，即使右侧段均已 SACK 确认，序号窗口也不能越过缺口。这保证接收窗口的固定空间约束，并通过单元测试和实际最大在途/缓存计数检查。

5.2 累计确认与位图映射

ACK 中的 ack_base 是 r，表示所有小于 r 的段都已按序写入；bit i 置 1 表示 $r+i$ 已收到并缓存在接收窗口。接收端先连续提交可交付前缀，再计算位图，所以 bit 0 必为 0。发送端据此清除已确认段的计时记录，只在连续确认前缀形成时推进 s。

例如丢失 DATA 2，而 3–7 已缓存：ack_base=2，SACK=`0x3E`，即位 1–5 置 1，分别对应 3–7。发送端只保留 2 为未确认；2 超时重传后，接收端一次交付 2–7，回复 ACK(base=8)。

表 4: 缺口场景下的选择确认解释

字段	数值	含义
ack_base	2	0、1 已按序提交，2 尚缺失
sack bit 0	0	分段 2 未收到
sack bits 1–5	1	分段 3–7 已缓存，不需重传
补齐后的 ack_base	8	0–7 全部提交

发送端还验证 ack_base 不得超过已发送前沿，SACK 不能包含未发送分段、bit 0 或超出 W 的位，避免错误确认导致跳过数据。

5.3 逐段超时与接收缓存

每个未确认段保存报文、最近发送时刻、重传次数。只有超过 RTO 且仍未确认的段才重传；已 SACK 的乱序段不会因为左侧缺口而被重传。这是选择重传，不是 Go-Back-N。固定 RTO 使用单调时钟，轮询及 Windows 调度会带来实际时间粒度，不保证毫秒配置等于精确链路时延。

接收端收到窗口内新段就缓存，随后从 r 开始连续写入；旧段或重复段只重发当前 ACK，不能重复写盘。窗口外但有效的段不缓存，只反馈当前窗口状态。缓存中的 SACK 段不是崩溃恢复承诺：本协议没有断点续传，进程失效后需要重新建立传输。

6 实现结构与核心方法

表 5: 模块与关键实现

模块	职责
wire.hpp	48 字节序列化、Internet 校验和、CRC32、报文约束
window.hpp	SendWindow 确认推进、ReceiveWindow 乱序缓存与按序交付
platform.hpp	基础 Winsock 函数、IPv4 地址、参数、UTF-8 路径与资源清理
transfer.cpp	发送/接收状态机、控制重试、逐段计时、临时文件提交、统计
relay.cpp	独立双向 UDP 故障转发，随机与定点故障
main.cpp	命令入口、退出码、最终 JSON 与统计文件
tests/	32 项协议测试、20 项集成测试、36 次性能与独立 SHA256 复核

6.1 基础 Socket 调用

启动调用 WSAStartup，创建 `socket(AF_INET,SOCK_DGRAM,IPPROTO_UDP)`，通过 bind 指定本机地址端口。每次 sendto 只发送一个完整 RUDP 报文；select 等待可读事件，recvfrom 一次取得一个 UDP 数据报及来源，之后校验实际长度与首部。最后 closesocket、WSACleanup。项目中用于生命周期的小型资源守卫仅负责释放原始句柄，不替代 Socket API 或协议实现。

6.2 发送端算法

```
while (base < total_segments) {
    while (next < total_segments && next < base + W)
        send_new_segment_and_start_its_timer();
    if (valid_ACK_received()) {
        mark_cumulative_prefix_and_SACK_bits();
        erase_acknowledged_timers();
        slide_only_contiguous_confirmed_prefix();
    }
    for (each still_unacknowledged_segment)
        if (its_timer_expired()) retransmit_only_that_segment();
}
```

源码实际使用有序集合管理未确认序号，并用映射保存未确认报文与独立计时。ACK 检查、推进与重传分别由明确函数负责。发送端预扫描文件得到大小与 CRC32，此时间不计入传输指标；实际传输按窗口流式读取，内存不随整个文件大小增长。

6.3 接收端与提交

```
insert_if_inside_receive_window(seq, payload);
while (buffer contains recv_base) {
    write_file_and_update_streaming_CRC(buffer[recv_base]);
    erase_buffered_segment();
    ++recv_base;
}
send_ACK(recv_base, bitmap_of_remaining_buffer);
```

只有写入成功后才增加 recv_base；写盘失败不会被错误地累计确认。临时文件用 Windows CREATE_NEW 创建，最终使用同目录重命名，不覆盖现有目标；异常析构只清理本进程创建且未提交的临时文件。UTF-16 命令行转 UTF-8，再经 filesystem 路径进入 Windows 文件 API，中文输入/输出文件名已覆盖。

7 功能与可靠性实测

7.1 测试方法与覆盖

测试时间为 2026-10-22 01:09:19 +0800，32 项协议/窗口测试与 20 项真实进程测试全部通过，集成过程耗时 10.421 秒。测试程序驱动真实 C++ 收发进程和 C++ 转发器；另外构造合法/非法对端报文，检查接收端错误路径。所有预期成功传输比较原文件与目标文件字节，并用 SHA256 独立核对。

32 项测试包含 RFC 校验向量、奇数字节、CRC 标准向量、单比特变异、首部字段边界、序号容量、固定窗口、无效 ACK、SACK bit63、乱序交付及随机窗口模型。20 项集成测试覆盖空文件、1 字节、1200/1201 边界、中英文与全部字节值、1MiB 二进制、W64、数据/ACK 丢失、损坏、乱序、重复、控制丢失、组合故障、窗口不符、重试耗尽、对端不可达、部分文件超时、CRC 失败、来源隔离和既有文件保护。

表 6: 代表性真实传输/故障场景

场景	重传	校验失败	乱序	文件结果
空文件	0	0	0	一致
1 MiB 二进制	0	0	0	一致
只丢 DATA 2	1	0	5	一致
末数据 ACK 丢失	1	0	0	一致
DATA 损坏	1	1	2	一致
首段延迟	0	0	7	一致
重复报文	0	0	0	一致
握手/关闭丢失	0	0	0	一致
组合随机故障	14	4	49	一致
窗口不一致	0	0	0	明确失败、无最终文件
全部 DATA 丢失	16	0	0	明确失败、无最终文件

“明确失败”是异常路径的预期结果，不是将不完整输出当作成功。部分超时与 CRC 不符均检查最终文件不存在、.part 已清理。

7.2 选择重传的实际日志

以下来自 2048 字节、W=8、chunk=256、只丢 DATA 2 的真实运行，完整输出另存于 evidence/cases。

```
STATE SYN_SENT session=e8fdc5bb51d9d874 bytes=2048 W=8
STATE ESTABLISHED pipeline/selective-repeat
RECV ACK base=1 sack=0x0
RECV ACK base=2 sack=0x0
RECV ACK base=2 sack=0x2
RECV ACK base=2 sack=0x6
RECV ACK base=2 sack=0xe
RECV ACK base=2 sack=0x1e
RECV ACK base=2 sack=0x3e
RETRANSMIT DATA seq=2 ack_base=0 sack=0x0
RECV ACK base=8 sack=0x0
STATE FIN_WAIT data complete; retransmissions=1
STATE TIME_WAIT receiver committed file, CRC32 verified
STATE CLOSED transfer successful
```

该次恰好重传一个 DATA 段，日志与 SACK 映射一致；不会重传已经缓存的 3–7。首段延后 60ms 但未丢失的另一场景，实际有 7 次乱序到达、最大缓存 8 段，DATA 重传为 0，说明乱序缓存不等于丢失。

7.3 控制阶段恢复

控制故障测试分别丢弃建立阶段的三条握手报文与就绪 ACK，以及关闭阶段的`FIN`、`FIN_ACK` 和`CLOSE_ACK`，最终双方仍正常关闭。这覆盖握手末确认丢失及关闭末确认丢失；日志中的 control_retries 与 DATA 重传分开统计，避免将握手重试混入数据重传率。重传预算耗尽时返回非零状态，对端收到 RESET 后终止；没有对端时 SYN 重试也会在预算内结束。

8 窗口与丢包性能实验

8.1 配置、计时与单位

使用同一份确定性生成的 128KiB 二进制文件（131072 字节），负载 1200 字节，总计 110 段。固定窗口 W=1、4、16、64；转发器对两个方向的每个数据报独立按 0%、10%、20% 概率丢弃；单向配置延迟 8ms，RTO100ms。每个组合用种子 2026、2027、2028 运行三次，共 36 次，全部 SHA256 一致。总执行耗时 107.282 秒，包括进程启动与 TIME_WAIT 等，不等于下表单次传输时间。

端到端计时从首条 SYN 发送前至有效 FIN_ACK 到达，包含建立、数据、重传、接收校验与文件提交，不包含发送端 CRC 预扫描及 TIME_WAIT。

$$G_{\mathrm{MiB/s}} = \frac{F}{2^{20}T}, \qquad G_{\mathrm{Mbit/s}} = \frac{8F}{10^{6}T}.$$

这是有效文件吞吐率，不包含首部与重复报文的线速。表中平均吞吐率是三次各自吞吐率的算术平均，不是平均时间的倒数。转发器的配置延迟不等于精确物理 RTT，Windows 定时与调度、文件刷新和本机负载都会影响结果。不同窗口虽使用相同种子，但报文到达顺序不同，因此不承诺丢失同一组序号。

8.2 实测表与图

表 7: 固定窗口与双向逐报文丢包概率；每格 3 次

丢包概率	窗口	平均时间/s	平均 MiB/s	MiB/s 范围	平均重传
0%	1	3.538	0.0353	0.0352–0.0355	0.0
0%	4	0.966	0.1294	0.1286–0.1308	0.0
0%	16	0.323	0.3866	0.3832–0.3887	0.0
0%	64	0.156	0.7997	0.7824–0.8321	0.0
10%	1	6.160	0.0203	0.0191–0.0212	23.3
10%	4	2.006	0.0627	0.0570–0.0691	12.3
10%	16	0.932	0.1348	0.1268–0.1474	12.0
10%	64	0.450	0.2805	0.2575–0.3205	9.3
20%	1	10.829	0.0116	0.0105–0.0127	66.0
20%	4	3.684	0.0344	0.0304–0.0404	41.3
20%	16	1.372	0.0918	0.0851–0.1035	28.7
20%	64	0.785	0.1740	0.1095–0.2087	24.0

原始 36 行数据在 `evidence/performance.csv`，每次的收发端与转发器统计保存在 benchmark 子目录，包含 SHA256、重传和窗口上界。

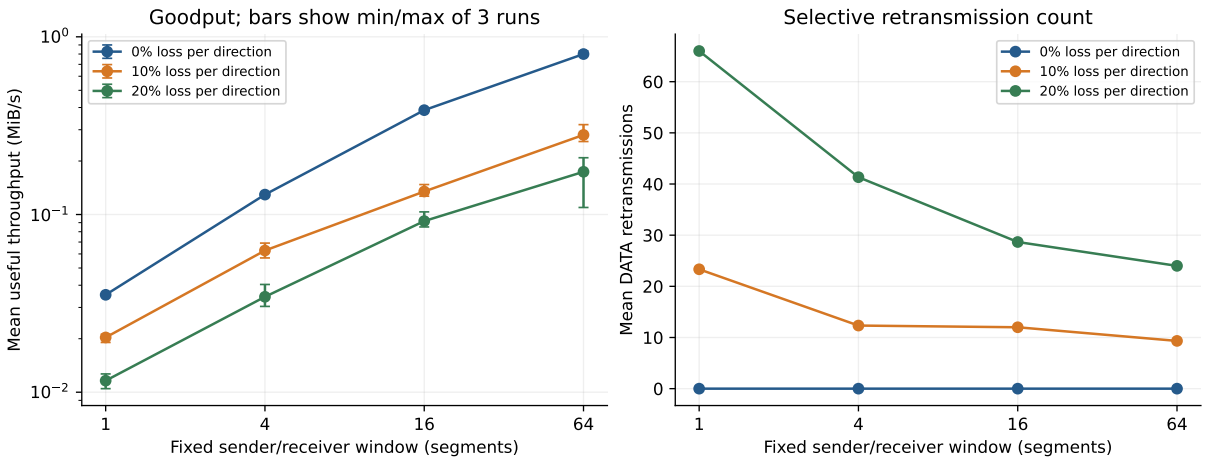


图 3: 平均有效吞吐率与 DATA 重传次数；吞吐率误差条表示 3 次观测的最小/最大值，并非置信区间

8.3 结果解释与局限

无人工丢包时，W=1 的平均有效吞吐率为 0.0353MiB/s，W=64 为 0.7997MiB/s，约提高 22.6 倍。20% 逐报文丢包时分别为 0.0116 与 0.1740MiB/s，约提高 15 倍。W=1 相当于停等，增大窗口使多段同时在途，提高等待确认期间的利用率。

同一窗口下丢包增加会降低吞吐率并增加重传；ACK 也可能丢失，因此“配置 20% 丢包”不能直接等同于 20%DATA 重传。较大窗口允许后续段先收到并 SACK 确认，缺口恢复后可连续提交；ACK 的累计覆盖还可能使个别 ACK 丢失无需重传数据。收益不是与 W 严格线性：握手、文件刷新、定时调度、重试及有限文件长度都带来固定开销。

W64、20% 组的吞吐范围约 0.1095–0.2087MiB/s，波动明显，控制报文丢失也会影响总时间。每组只有 3 次，本次结果可用于说明窗口影响，不能宣称统计显著或外推到任意真实广域网。本协议没有拥塞控制，窗口增大不代表在共享网络上可以无限提升发送速率；现场仍应按课堂规定环境重测。

9 问题处理与结论

1. **UDP 不提供连接状态。**用显式会话 ID、地址检查、状态机、幂等重试和 RESET 建立应用层连接管理。
2. **仅累计确认无法表达缓存的乱序段。**增加 64 位 SACK，按 ack_base 相对序号映射，逐段停止计时。
3. **只按未确认数量开窗可能越界。**以最左缺口和下一序号之差限制固定序号空间，而非仅看容器大小。
4. **收到不等于完整交付。**窗口内缓存，按序写入，再做长度和整文件 CRC 验证；提交前保留.part 状态。
5. **丢失关闭确认可能造成状态分歧。** FIN 幂等处理、FIN_ACK 重发、发送 TIME_WAIT；超过预算明确说明最终文件与关闭确认状态。
6. **性能数值容易口径混乱。**分清端到端、数据阶段、预扫描与 TIME_WAIT，保留原始三次值、统一 MiB/s 单位。

本实验完成基于基础 UDP 套接字的单向可靠文件传输：连接管理、全报文差错检测、流水线选择确认、逐段重传与同大小固定窗口均已实现。32 项协议测试、20 项集成测试和 36 次性能传输提供了可复核证据。协议边界包括有限重试、无身份认证/加密、无拥塞控制和无断点续传；本机模拟结果与课堂待指定环境明确区分。

参考文献

[1] Braden R., Borman D., Partridge C. Computing the Internet Checksum. RFC 1071, 1988. https://www.rfc-editor.org/rfc/rfc1071.

[2] Microsoft. Using Winsock. https://learn.microsoft.com/en-us/windows/win32/winsock/using-winsock.

[3] 课程教学材料：《实验说明》，编程实验 3 与演示讲解要求，第 6、8 页。2026 年课程提供版本。